

Relatório 6 - Prática de formulários e dashboard

Kaíque Medeiros Lima

1. Descrição da atividade

a. Formulários com React Hook Form

A atividade continua o projeto de Fórmula 1 com foco em formulários e na experiência de uso de um dashboard. As telas de login, cadastro e perfil usam React Hook Form para registrar os campos, tratar o envio e acessar os erros. Cada campo é conectado ao formulário pelo register, e o handleSubmit só chama a função de envio quando todos os valores passam na validação.

O useForm recebe o tipo dos dados e o zodResolver, que liga o formulário às regras do Zod. No login, os campos são e-mail e senha; no cadastro, entram também o nome e a confirmação de senha; no perfil, apenas o nome. Os formulários usam noValidate para que a validação fique por conta dessa integração, com as mensagens no mesmo padrão visual em toda a aplicação.

b. Schemas e validação dos dados

As regras ficam em arquivos schemas.ts separados das telas, um no módulo de autenticação e outro no módulo de perfil. O loginSchema exige um e-mail válido e uma senha com pelo menos seis caracteres. O registerSchema acrescenta um nome com no mínimo três caracteres e a confirmação da senha, e o refine compara as duas senhas, associando o erro ao campo confirmPassword quando elas são diferentes. Para o e-mail foi usado z.email, que é a forma recomendada no Zod 4 no lugar do encadeamento z.string().email, que ficou depreciado.

Os tipos ILoginSchema, IRegisterSchema e IProfileSchema são obtidos com z.infer a partir dos próprios schemas, então o formato usado pelo TypeScript acompanha a definição da validação. A regra de três caracteres para o nome verifica apenas o tamanho do texto, não se ele representa um nome completo.

```
CARD 06 / modules/auth/schemas.ts

1 export const registerSchema = z
2   .object({
3     name: z.string().trim().min(3, "Informe pelo menos 3 caracteres"),
4     email: z.email("Informe um e-mail válido"),
5     password: z.string().min(6, "A senha deve ter no mínimo 6 caracteres"),
6     confirmPassword: z.string().min(1, "Confirme sua senha"),
7   })
8   .refine((data) => data.password === data.confirmPassword, {
9     message: "As senhas precisam ser iguais",
10    path: ["confirmPassword"],
11  });
12
13 export type IRegisterSchema = z.infer<typeof registerSchema>;
```

Figura 1 - Schema de cadastro e comparação das senhas. Código da branch do Card 06.

c. Mensagens de erro e componentes compartilhados

O componente `TextField` recebe uma mensagem de erro opcional e a exibe em vermelho abaixo do campo, ligada ao input por `aria-describedby` e com `aria-invalid` quando há problema. As três telas de formulário e o formulário de métricas do dashboard reaproveitam esse componente, o que mantém o visual e a acessibilidade iguais sem repetir estilos.

Além dos erros de cada campo, há erros que só aparecem depois do envio. No login, quando o e-mail e a senha passam no schema mas não correspondem a nenhuma conta, a tela mostra a mensagem E-mail ou senha incorretos em um elemento com `role="alert"`. No cadastro, um e-mail já usado gera a mensagem de conta existente. A versão anterior simulava a espera com `setTimeout`; como a verificação agora é feita localmente e de forma síncrona, esse atraso artificial foi removido.

d. Autenticação simulada com Zustand

O estado de autenticação fica em uma store do Zustand, `useAuthStore`, com o middleware `persist` salvando o usuário logado e as contas no `localStorage`. A função `signIn` procura uma conta com o e-mail e a senha digitados e devolve verdadeiro ou falso, e a tela de login decide entre mostrar o erro e redirecionar para `/dashboard`. Existe uma conta fixa de demonstração, `piloto@f1.com` com a senha `senha123`, documentada no README e exibida na própria tela de login.

O cadastro também passou a usar os dados de verdade: a função `register` adiciona a nova conta à store, recusa e-mails repetidos e já deixa o usuário logado. Depois de sair, é possível entrar de novo com a conta criada. Como não há backend, as contas ficam apenas no navegador, com a senha em texto no `localStorage`, o que serve somente para a simulação e não deve ser usado com senhas reais.

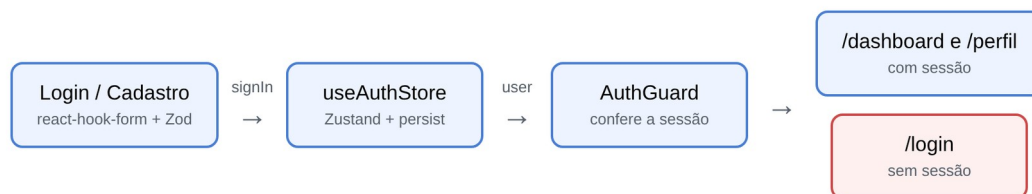


Figura 2 - Fluxo da autenticação simulada até as rotas protegidas. Autoria própria.

CARD 06 / modules/auth/store.ts (trechos)

```
1   signIn: ({ email, password }) => {
2     const account = get().accounts.find(
3       (item) => item.email === email.toLowerCase() && item.password === password,
4     );
5
6     if (!account) return false;
7
8     set({ user: { name: account.name, email: account.email } });
9     return true;
10  },
11
12
13  // ...
14  {
15    name: "f1-auth",
16    // hidrata só no cliente, depois da montagem, para o HTML do servidor bater com o primeiro render
17    skipHydration: true,
18    partialize: ({ user, accounts }) => ({ user, accounts }),
19    onRehydrateStorage: () => () => useAuthStore.setState({ hydrated: true }),
20  },
21  ),
```

Figura 3 - Função signIn e configuração do persist na store de autenticação. Código da branch do Card 06.

Figura 4 - Login com senha incorreta mostrando a mensagem de erro. Autoria própria, navegador.

e. Proteção de rotas e saída

As rotas /dashboard e /perfil são protegidas pelo componente AuthGuard, um Client Component que lê a store e, quando não há usuário, redireciona para /login com router.replace. As páginas continuam como Server Components: elas apenas envolvem a tela com o AuthGuard, que é quem precisa rodar no navegador. Enquanto a sessão não é verificada, o guard mostra uma mensagem de carregamento em vez do conteúdo protegido.

A principal dificuldade foi a hidratação. O servidor não tem acesso ao localStorage, então se a store fosse restaurada logo ao carregar, o primeiro render no navegador seria diferente do HTML gerado no servidor. Por isso a store usa skipHydration e o hook useAuthHydrated chama rehydrate depois da montagem, marcando hydrated quando termina. O cabeçalho usa o mesmo hook para mostrar o nome do usuário e o botão Sair, que limpa a sessão e leva para /login. O nome alterado no perfil é salvo na store com updateName e aparece no cabeçalho na hora.

```

CARD 06 / modules/auth/components/AuthGuard

1  export const AuthGuard = ({ children }: AuthGuardProps) => {
2    const hydrated = useAuthHydrated();
3    const user = useAuthStore((state) => state.user);
4    const router = useRouter();
5
6    useEffect(() => {
7      if (hydrated && !user) router.replace("/login");
8    }, [hydrated, user, router]);
9
10   if (!hydrated || !user) {
11     return (
12       <p role="status" className="text-gray-600">
13         {hydrated ? "Redirecionando para o login..." : "Verificando sessão..."}
14       </p>
15     );
16   }
17
18   return children;
19 };

```

Figura 5 - Componente AuthGuard. Código da branch do Card 06.

F1 2021 - Dashboard

Início Dashboard Perfil **Olá, Kaique** Sair

Meu perfil

Troque seu nome

Nome

Kaique

Salvar nome

Nome salvo: Kaique (piloto@f1.com)

Figura 6 - Perfil com o nome salvo refletido no cabeçalho. Autoria própria, navegador.

F1 2021 - Dashboard

Início Dashboard Perfil Login **Cadastro**

Login

Entre para ver dados da temporada de Fórmula 1.

E-mail

voce@email.com

Senha

Mínimo 6 caracteres

Entrar

Conta de demonstração: piloto@f1.com / senha123

[Criar uma conta](#)

Figura 7 - Tela de login depois de clicar em Sair, com o cabeçalho sem sessão. Autoria própria, navegador.

f. Dashboard com gráficos e filtro

A rota /dashboard mostra a temporada de 2021 com indicadores no topo, um gráfico de barras com os pontos de cada piloto na classificação final e um gráfico de linhas com a evolução dos três primeiros colocados corrida a corrida, os dois feitos com recharts. Abaixo ficam a classificação em lista, o destaque do campeão e as últimas corridas. Os indicadores não são escritos à mão: a função buildSeasonMetrics calcula o piloto com mais vitórias, os pontos do líder e a vantagem sobre o segundo a partir das corridas já disputadas.

O filtro por equipe é um select que altera o estado `selectedTeam`. A lista de pilotos e o gráfico de barras usam a mesma lista filtrada, calculada com `useMemo`, então os dois mudam juntos assim que outra equipe é escolhida. Quando o filtro não encontra pilotos, a tela mostra uma mensagem em vez de um gráfico vazio. O layout usa `grid` e classes responsivas do `Tailwind` para organizar os blocos em telas menores e maiores.

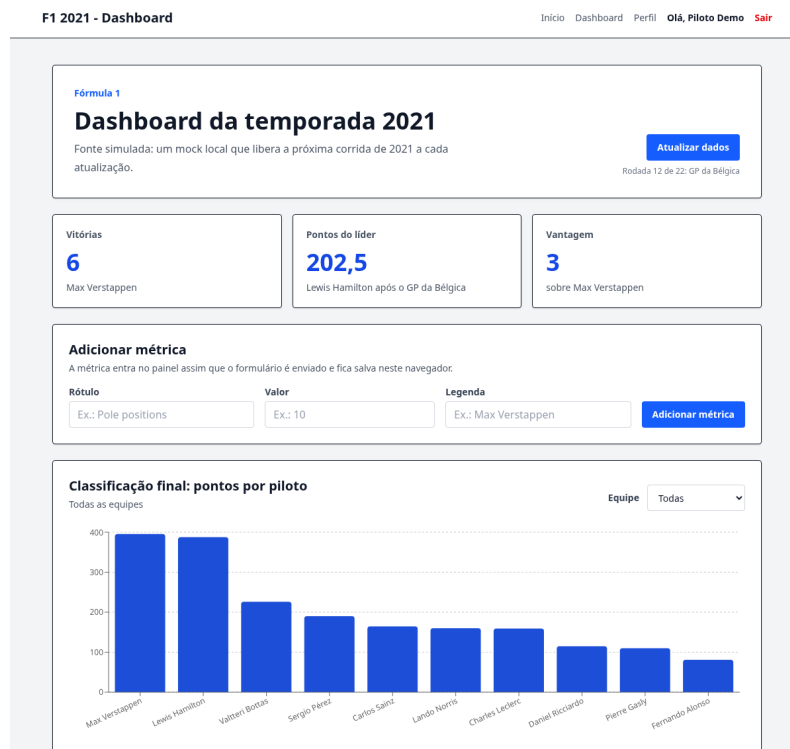


Figura 8 - Topo do dashboard com indicadores, formulário de métrica e gráfico de barras. Autoria própria, navegador.

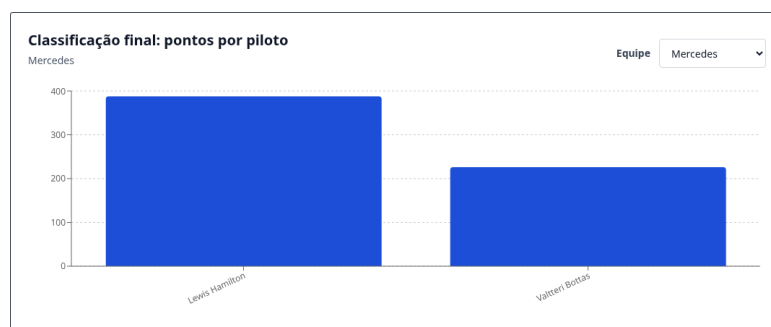


Figura 9 - Gráfico de barras filtrado pela equipe Mercedes. Autoria própria, navegador.

g. Atualização dos dados

Os dados vêm de `fetchDashboardData`, que simula um endpoint com um atraso de 600 ms e devolve a temporada até a rodada pedida. O dashboard começa na rodada 12, e cada clique em `Atualizar dados` busca a rodada seguinte: os indicadores, o gráfico de evolução e as últimas corridas mudam de acordo com o novo resultado, e a tela informa a rodada atual e a corrida mais recente. Ao chegar à rodada 22, o botão fica desabilitado com o texto `Temporada completa`. O texto do cabeçalho deixa claro que a fonte é simulada.

Os pontos acumulados usados na evolução são os dos três primeiros colocados de 2021, e as últimas corridas passaram a ser derivadas da mesma lista de corridas, sem uma

segunda lista duplicada. A classificação completa dos dez pilotos existe apenas no resultado final, por isso o gráfico de barras é identificado como classificação final. A busca usa AbortController: a carga inicial é cancelada ao desmontar a tela, e cada atualização cancela a anterior e também é cancelada se o componente sair da tela, evitando atualizar o estado de um componente desmontado.

```
CARD 06 / DashboardScreen (atualização)

1  useEffect(() => () => refreshController.current?.abort(), []);
2
3  const handleRefresh = async () => {
4    refreshController.current?.abort();
5    const controller = new AbortController();
6    refreshController.current = controller;
7
8    setIsRefreshing(true);
9    setError(null);
10
11    try {
12      const result = await fetchDashboardData(
13        data ? data.round + 1 : INITIAL_ROUND,
14        controller.signal,
15      );
16      setData(result);
17    } catch (refreshError) {
18      if (!isAbortError(refreshError)) {
19        setError("Não foi possível atualizar os dados.");
20      }
21    } finally {
22      if (!controller.signal.aborted) setIsRefreshing(false);
23    }
24  };
```

Figura 10 - Atualização dos dados com cancelamento pelo AbortController. Código da branch do Card 06.

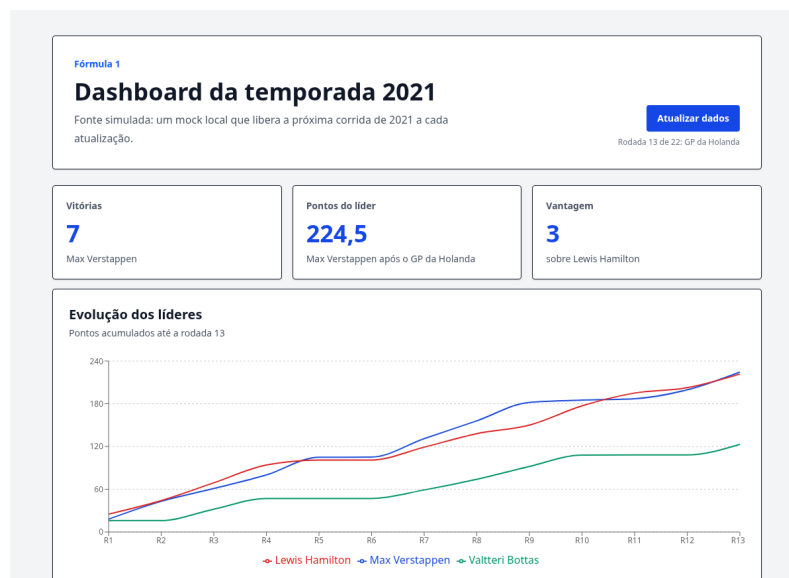


Figura 11 - Dashboard depois de uma atualização, na rodada 13, com indicadores e evolução atualizados. Autoria própria, navegador.

h. Métricas personalizadas

O componente MetricForm usa React Hook Form com o metricSchema para adicionar uma métrica com rótulo, valor e legenda. O valor aceita vírgula como separador decimal e

precisa ser um número maior que zero. As métricas adicionadas ficam em uma segunda store do Zustand, useMetricsStore, também persistida, então continuam no painel ao sair e voltar para o dashboard ou recarregar a página.

Cada métrica recebe um id gerado com crypto.randomUUID no momento em que é adicionada. Esse id é usado como key na lista e para remover a métrica certa pelo botão Remover, em vez de depender da posição no array ou da referência do objeto, que podiam falhar com métricas de mesmo conteúdo.

The screenshot shows a dashboard with three metric cards: 'Vitórias' (7), 'Pontos do líder' (224,5), and 'Vantagem' (3). Below these is a 'Pole positions' card (10) with a 'Remover' button. At the bottom is a form titled 'Adicionar métrica' with fields for 'Rótulo' (X), 'Valor' (Ex.: 10), and 'Legenda' (Ex.: Max Verstappen). Red error messages are visible below the 'Rótulo' and 'Legenda' fields: 'Informe pelo menos 3 caracteres'.

Figura 12 - Métrica adicionada pelo formulário e erros de validação em um novo envio. Autoria própria, navegador.

i. Testes e verificação

Os testes usam Vitest. Além dos testes dos schemas de login, cadastro e métrica, foram criados testes para a store de autenticação, cobrindo login certo e errado, cadastro seguido de novo login, e-mail repetido e troca de nome, para a store de métricas, cobrindo ids únicos, persistência e remoção pelo id, e para buildSeasonMetrics e fetchDashboardData, conferindo os indicadores na rodada 5 e no fim da temporada. Os testes das stores rodam com o ambiente jsdom para ter acesso ao localStorage. São 21 testes em 5 arquivos, todos passando, e o projeto também passa no lint, no TypeScript e no build de produção.

```
$ npm test
RUN v5.0.1 forms-dashboard
Test Files 5 passed (5)
Tests 21 passed (21)
Duration 1.30s (tests 60%, environment 31%, import 5%, transform 4%, worker 1%)
```

Figura 13 - Execução dos testes. Autoria própria, terminal.

2. Conclusões

O card mostrou que um formulário precisa cuidar tanto dos dados quanto da resposta ao usuário, e que o React Hook Form com Zod resolve bem as duas coisas. No dashboard, a diferença entre mostrar números em cards e visualizar dados ficou clara ao trocar listas por gráficos e ligar um filtro que muda o que aparece na hora. O Zustand organizou o estado global de sessão e de métricas com pouco código, mas exigiu cuidado com a hidratação no Next.js e com o cancelamento das buscas. O limite do projeto é não ter backend: a autenticação e a atualização são simuladas, o que deixa a base pronta para trocar o mock por uma API real.

3. Referências

<https://react-hook-form.com/>

<https://zod.dev/>

<https://zustand.docs.pmnd.rs/integrations/persisting-store-data>

<https://recharts.org/>

<https://developer.mozilla.org/en-US/docs/Web/API/AbortController>

https://www.youtube.com/watch?v=er_QPBldsXE

<https://www.youtube.com/watch?v=PqRe0Q9b5QI>